Take-Home Engineering Assignment

RAG Ingestion Pipeline for Enterprise Document Intelligence

> **Estimated Time:** 4–6 hours
> **Submission:** A GitHub repository link with a working implementation, a `DESIGN.md`, and instructions to run the project locally.

Background

You are joining a team building a **Retrieval-Augmented Generation (RAG)** platform for enterprise customers. The platform ingests large volumes of heterogeneous documents — PDFs, Word files, HTML, spreadsheets, scanned forms — and makes their content retrievable by a downstream answer-generation service.

Enterprise customers upload insurance benefit books, regulatory filings, and clinical protocols. A malformed chunk or a missed table can produce a legally significant wrong answer.

The Task

Build a production-grade **document ingestion pipeline** for a RAG system.

Your implementation should expose (at minimum) the following interface:

```
async def ingest(
    file_bytes: bytes,
    filename: str,
    document_id: str,
    tenant_id: str,
    collection_id: str,
    document_type: str | None,
    tags: list[str],
    metadata: dict,
) -> IngestResult:
    ...
```

The system should be designed as if it will process **millions of documents** from hundreds of enterprise tenants in production. The architecture decisions you make — and why — are as important as the working code.

Parser Output Reference

Your pipeline will need to handle output from at least one of the two parsers below. Each produces a different JSON shape for the same source document. You may choose which one to implement an adapter for; supporting both is a bonus.

Parser A — Azure Document Intelligence (ADI)


```
{
  "paragraphs": [
    {
      "content": "All amounts shown are in-network rates.",
      "role": null,
      "bounding_regions": [{ "page_number": 1, "polygon": [0.5, 0.7, 4.2, 0.7,
4.2, 0.9, 0.5, 0.9] }],
      "spans": [{ "offset": 0, "length": 40 }]
    },
    {
      "content": "Annual Deductible",
      "role": "sectionHeading",
      "bounding_regions": [{ "page_number": 1, "polygon": [0.5, 1.0, 3.0, 1.0,
3.0, 1.2, 0.5, 1.2] }],
      "spans": [{ "offset": 41, "length": 17 }]
    }
```

```
  ],
  "tables": [
    {
      "row_count": 3,
      "column_count": 2,
      "cells": [
        { "row_index": 0, "column_index": 0, "content": "Service", "kind":
"columnHeader" },
        { "row_index": 0, "column_index": 1, "content": "Your Cost", "kind":
"columnHeader" },
        { "row_index": 1, "column_index": 0, "content": "Primary Care Visit",
"kind": "content" },
        { "row_index": 1, "column_index": 1, "content": "$20 copay", "kind":
"content" },
        { "row_index": 2, "column_index": 0, "content": "Specialist Visit",
"kind": "content" },
        { "row_index": 2, "column_index": 1, "content": "$40 copay", "kind":
"content" }
      ],
      "bounding_regions": [{ "page_number": 1 }],
      "spans": [{ "offset": 58, "length": 200 }],
      "caption": { "content": "Table 1: Cost Sharing Summary" }
    }
  ],
  "figures": [
    {
      "bounding_regions": [{ "page_number": 2, "polygon": [1.0, 3.0, 5.0, 3.0,
5.0, 6.5, 1.0, 6.5] }],
      "spans": [{ "offset": 350, "length": 0 }],
      "caption": { "content": "Figure 1: Network Coverage Map" }
    }
  ],
  "pages": [
    { "page_number": 1, "width": 8.5, "height": 11.0, "spans": [{ "offset": 0,
"length": 600 }] }
  ],
  "content": "All amounts shown are in-network rates.\nAnnual Deductible\n..."
}
```

Parser B — Docling

```
{
  "schema_name": "DoclingDocument",
  "version": "1.3.0",
  "name": "benefit_summary",
  "origin": {
    "mimetype": "application/pdf",
    "binary_hash": "a3f5...",
    "filename": "benefit_summary.pdf"
  },
  "body": {
    "self_ref": "#/body",
    "children": [
      { "$ref": "#/texts/0" },
      { "$ref": "#/texts/1" },
      { "$ref": "#/tables/0" }
    ],
    "label": "unspecified"
  },
  "texts": [
    {
      "self_ref": "#/texts/0",
      "parent": { "$ref": "#/body" },
      "label": "section_header",
      "prov": [{ "page_no": 1, "bbox": { "l": 36, "t": 700, "r": 400, "b": 720,
"coord_origin": "BOTTOMLEFT" }, "charspan": [0, 17] }],
      "text": "Annual Deductible"
    },
    {
      "self_ref": "#/texts/1",
      "parent": { "$ref": "#/body" },
      "label": "text",
      "prov": [{ "page_no": 1, "bbox": { "l": 36, "t": 670, "r": 540, "b": 690,
"coord_origin": "BOTTOMLEFT" }, "charspan": [18, 95] }],
      "text": "All amounts shown are in-network rates."
    }
  ],
  "tables": [
    {
      "self_ref": "#/tables/0",
      "parent": { "$ref": "#/body" },
      "label": "table",
```

```
      "prov": [{ "page_no": 1, "bbox": { "l": 36, "t": 400, "r": 540, "b": 650,
"coord_origin": "BOTTOMLEFT" } }],
      "data": {
        "table_cells": [
          { "start_row_offset_idx": 0, "end_row_offset_idx": 1,
"start_col_offset_idx": 0, "end_col_offset_idx": 1, "text": "Service",
"column_header": true },
          { "start_row_offset_idx": 0, "end_row_offset_idx": 1,
"start_col_offset_idx": 1, "end_col_offset_idx": 2, "text": "Your Cost",
"column_header": true },
          { "start_row_offset_idx": 1, "end_row_offset_idx": 2,
"start_col_offset_idx": 0, "end_col_offset_idx": 1, "text": "Primary Care
Visit", "column_header": false },
          { "start_row_offset_idx": 1, "end_row_offset_idx": 2,
"start_col_offset_idx": 1, "end_col_offset_idx": 2, "text": "$20 copay",
"column_header": false }
        ],
        "num_rows": 2,
        "num_cols": 2
      },
      "captions": [{ "$ref": "#/texts/5" }]
    }
  ],
  "pictures": [
    {
      "self_ref": "#/pictures/0",
      "parent": { "$ref": "#/body" },
      "label": "picture",
      "prov": [{ "page_no": 2, "bbox": { "l": 72, "t": 300, "r": 540, "b": 580,
"coord_origin": "BOTTOMLEFT" } }],
      "captions": [{ "$ref": "#/texts/6" }]
    }
  ],
  "pages": {
    "1": { "size": { "width": 612, "height": 792 } },
    "2": { "size": { "width": 612, "height": 792 } }
  }
}
```

Deliverables

Deliverable	Required
Working implementation	✅
`DESIGN.md`	✅
Unit tests	✅
A `docker-compose.yml` or local run script	✅
Second parser adapter	➕ Bonus

Your `DESIGN.md` should explain the key architectural decisions you made, the trade-offs you considered, and what you would tackle next.

Constraints

Language: Python preferred; Go or TypeScript acceptable with justification.
Vector store: Any open-source option (Qdrant, Weaviate, pgvector, etc.). Mock it in tests.
Embeddings: Any open model or API. Mock in tests.

Evaluation

Code quality, architectural clarity, and the reasoning in your `DESIGN.md` are weighted equally with working functionality. We are looking for engineers who think about systems, not just scripts.

Questions? Email [hiring@example.com]. Please do not publicly post this assignment or your solution while interviewing.